

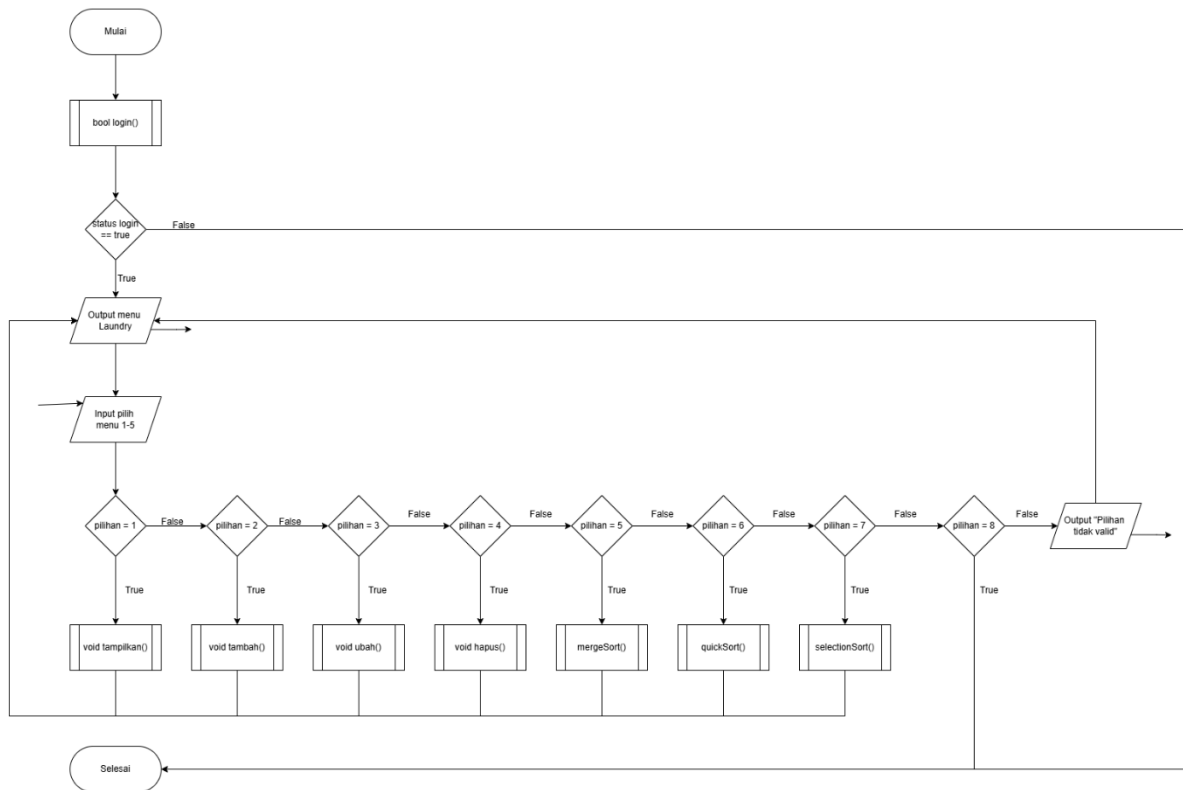
LAPORAN PRAKTIKUM
POSTTEST 5
ALGORITMA PEMROGRAMAN LANJUT



Disusun oleh:
Zeinal Abidin (2509106075)
Kelas (B2 '25)

PROGRAM STUDI INFORMATIKA
UNIVERSITAS MULAWARMAN
SAMARINDA
2026

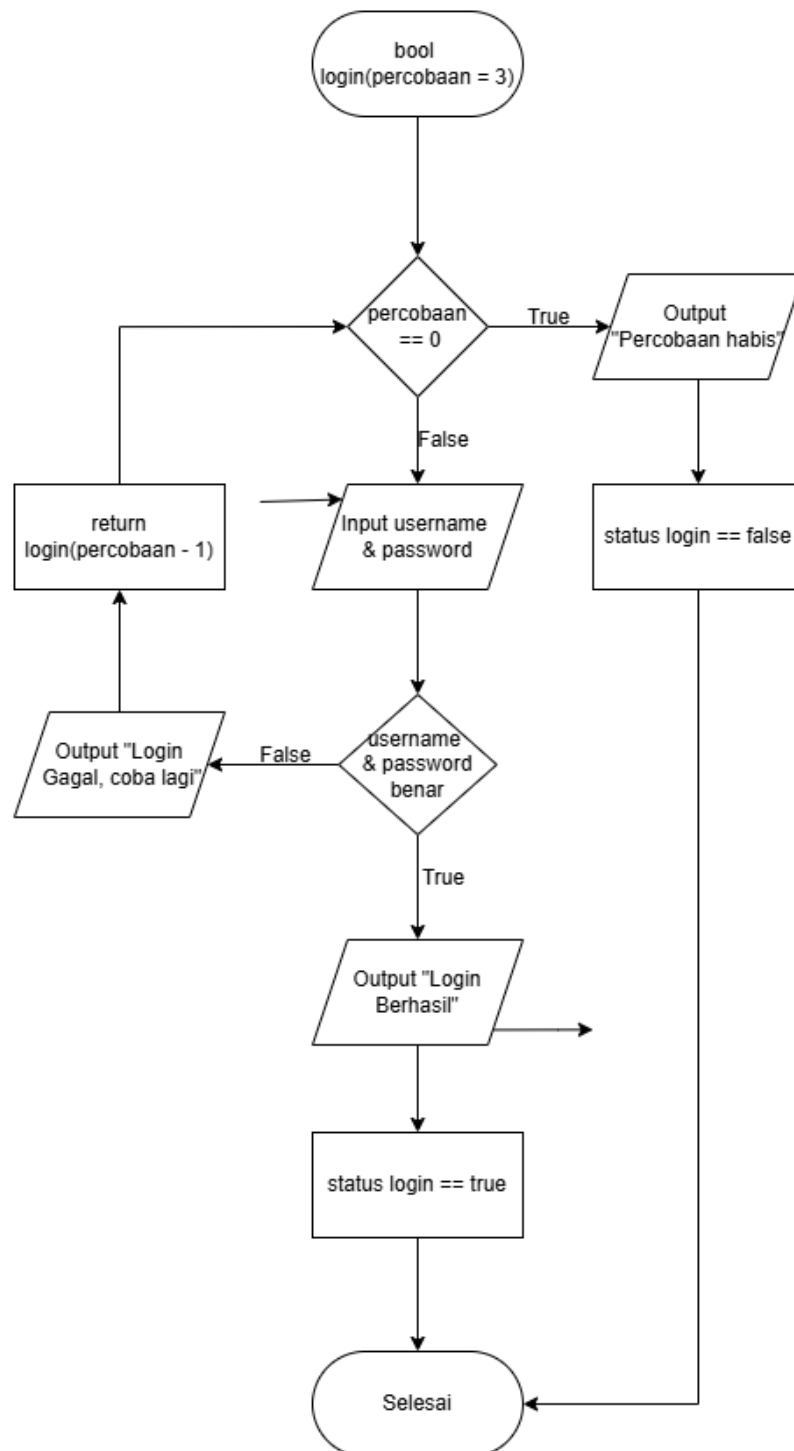
1. Flowchart



Gambar 1.1 Flowchart.Main

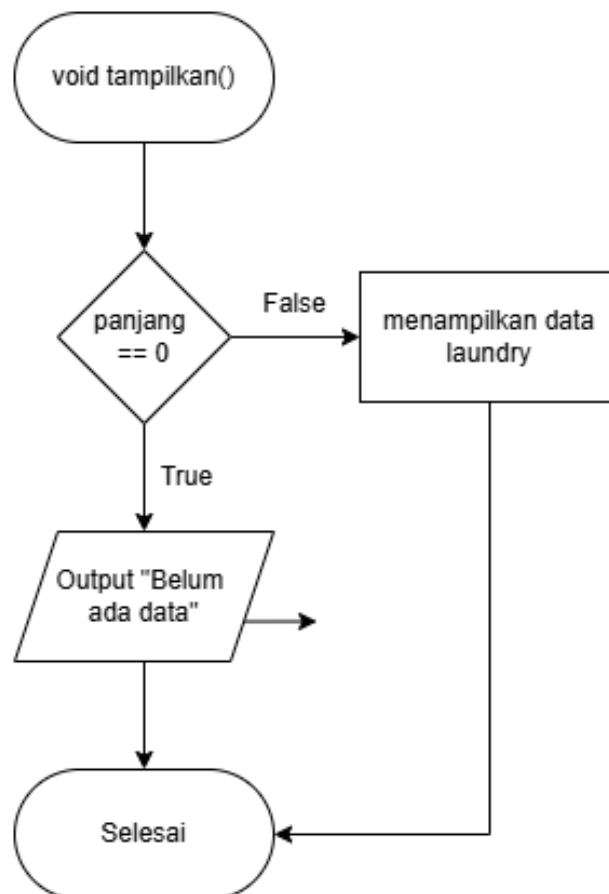
- Mulai : Mulai menandakan dimulainya flowchart.
- Decision Login :
 - a. Jika return menghasilkan true maka program menampilkan menu laundry.
 - b. Jika return tidak menghasilkan true maka program berhenti.
- Decision Menu laundry :
 - a. Menampilkan menu laundry.
 - b. Input pilih menu.
 - c. Jika pilihan = 1, maka program akan mengeksekusi fungsi tampilkan data.
 - d. Jika pilihan = 2, maka program akan mengeksekusi fungsi tambah data.
 - e. Jika pilihan = 3, maka program akan mengeksekusi fungsi ubah data.
 - f. Jika pilihan = 4, maka program akan mengeksekusi fungsi hapus data.
 - g. Jika pilihan = 5, maka program akan menampilkan data sebelum dan sesudah di merge sorting.
 - h. Jika pilihan = 6, maka program akan menampilkan data sebelum dan sesudah di quick sorting.

- i. Jika pilihan = 7, maka program akan menampilkan data sebelum dan sesudah di selection sorting.
- j. Jika pilihan = 8, maka program berhenti.
- Selesai : Selesai menandakan akhir dari flowchart.



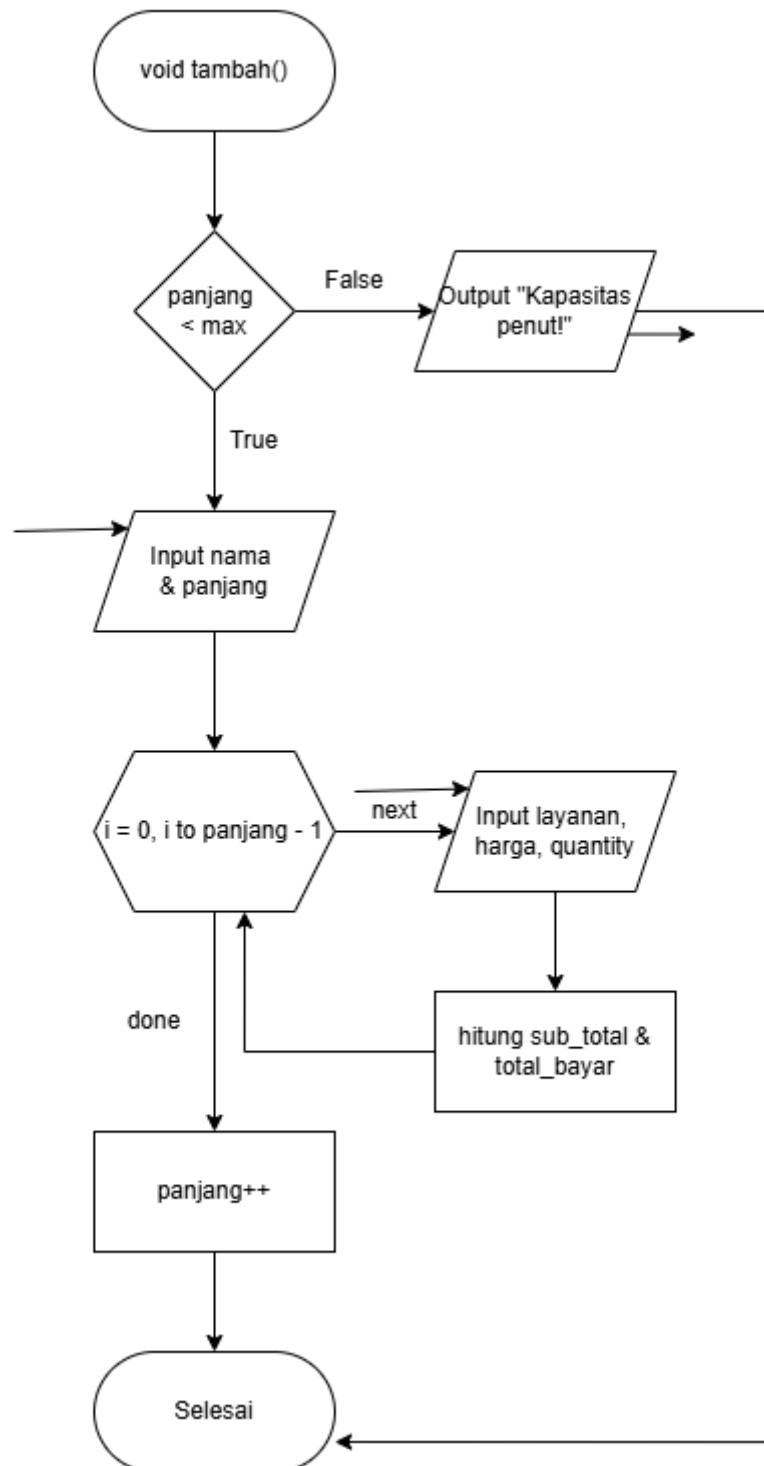
Gambar 1.2 Flowchart Fungsi Login.

- login(percobaan = 3): Fungsi menerima parameter percobaan dengan nilai default 3.
- Kondisi Berhenti: Hal pertama yang dilakukan sistem adalah mengecek apakah percobaan == 0.
 - Jika true: Sistem langsung mengeluarkan output “Percobaan habis.”, mereturn false dan selesai.
- Proses Input: Jika sisa percobaan masih ada, pengguna diminta memasukkan username dan password.
- Validasi:
 - Jika benar: Muncul output “Login berhasil”, fungsi mereturn true dan selesai.
 - Jika salah: Muncul output “Login gagal, coba lagi”.
- Pemanggilan Rekursif: Setelah gagal, sistem memanggil kembali fungsi itu sendiri namun dengan parameter percobaan – 1.
- Selesai : Selesai menandakan akhir dari flowchart.



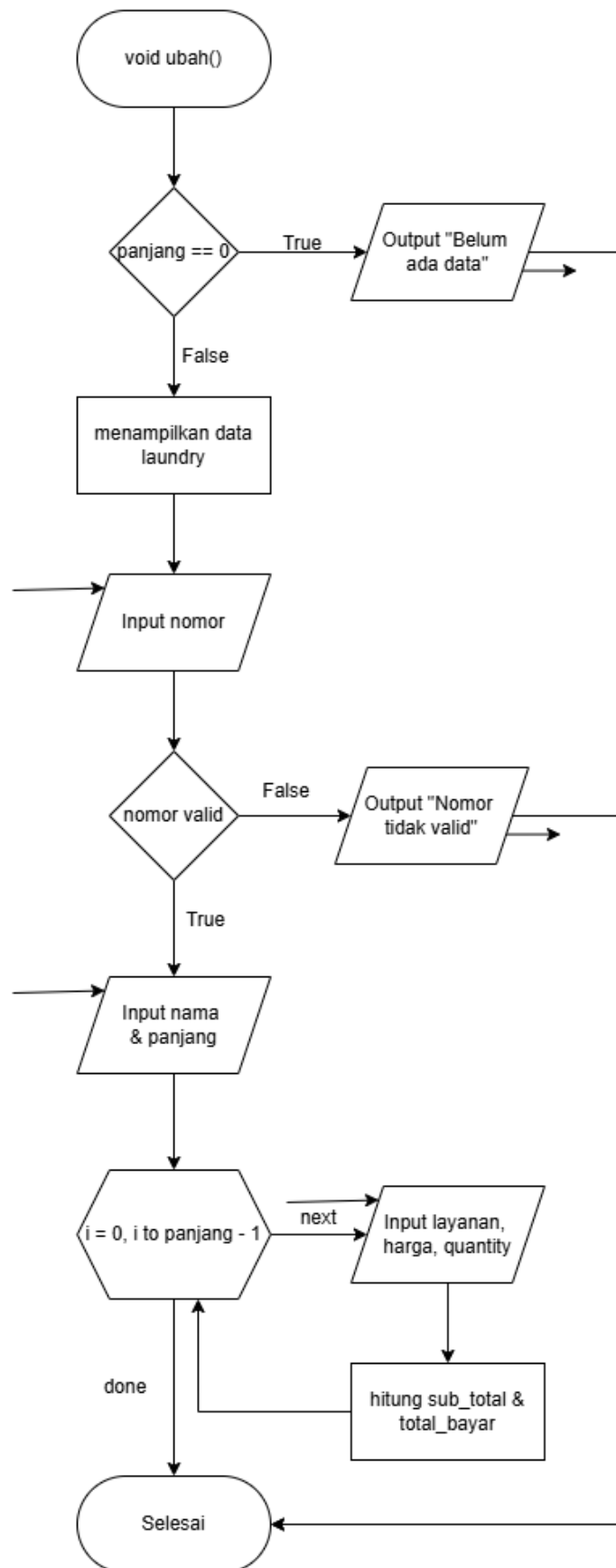
Gambar 1.3 Flowchart Fungsi Tampilkan.

- tampilkan(): Awal dimulainya flowchart fungsi tampilkan data.
- Validasi Awal: Mengecek apakah panjang == 0.
 - Jika true: Muncul output “Belum ada data”.
 - Jika false: Maka program akan menampilkan seluruh data.
- Selesai : Selesai menandakan akhir dari flowchart.



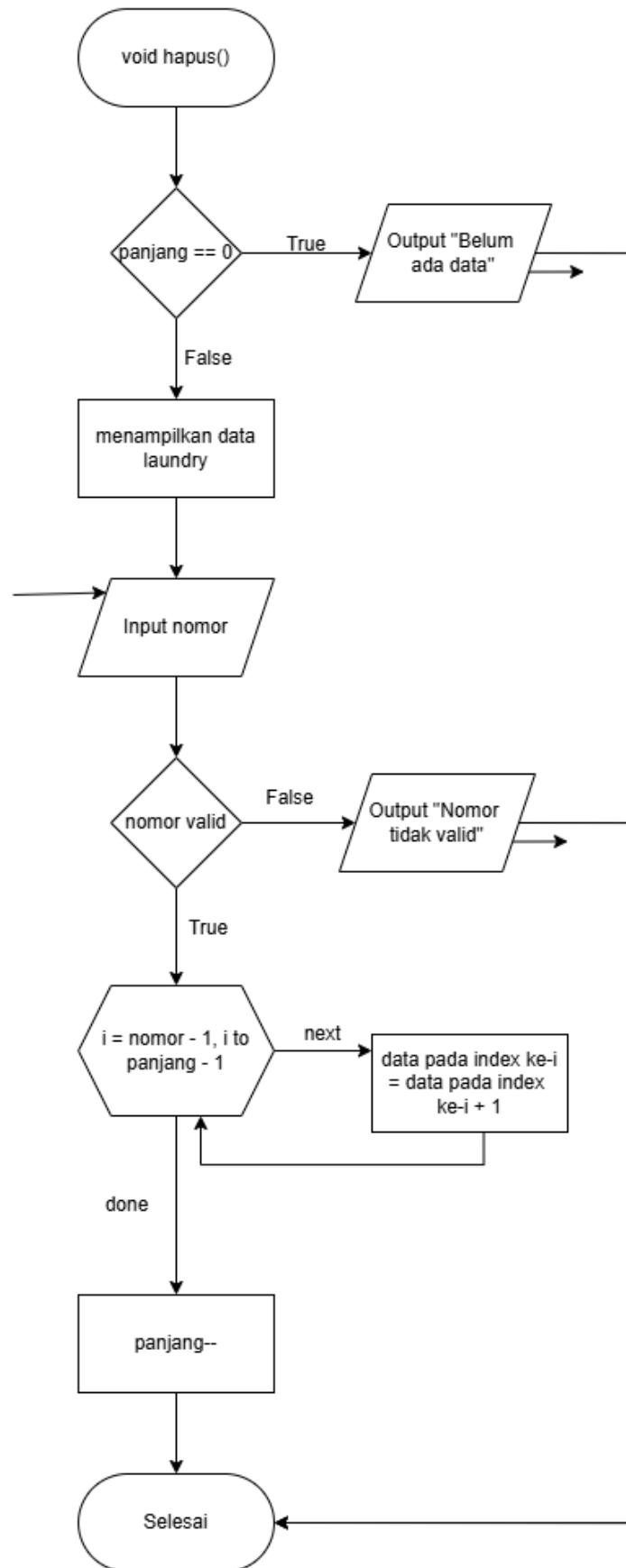
Gambar 1.4 Flowchart Fungsi Tambah.

- tambah(): Awal dimulainya flowchart fungsi tambah data.
- Validasi Awal: Mengecek apakah panjang < max. Jika false maka muncul output “Kapasitas Penuh”. Jika true maka lanjut input data.
- Input Data: Pengguna memasukkan nama dan jumlah item.
- Looping: Menggunakan perulangan untuk menginput detail layanan, harga dan kuantitas, lalu menghitung subtotal dan total bayar.
- Finalisasi: Setelah loop selesai variabel panjang bertambah satu.
- Selesai : Selesai menandakan akhir dari flowchart.



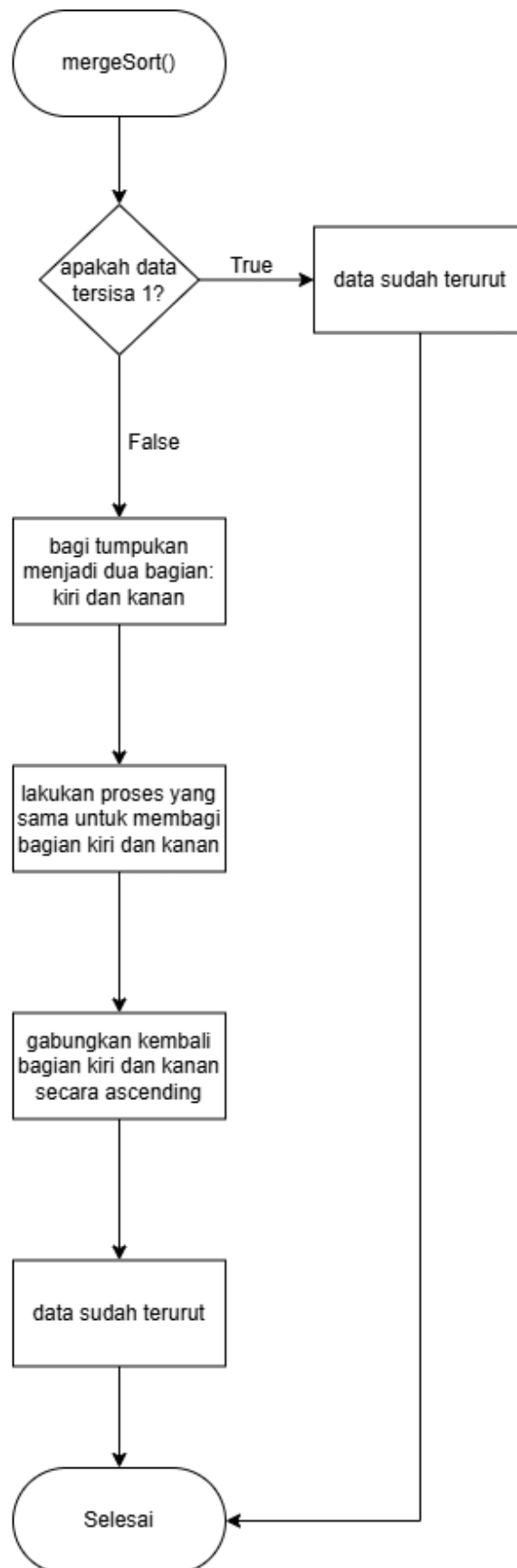
Gambar 1.5 Flowchart Fungsi Ubah.

- ubah(): Awal dimulainya flowchart fungsi ubah data.
- Validasi Awal: Mengecek apakah panjang == 0. Jika true maka muncul output “Belum ada data”. Jika false maka tampilkan data dan minta pengguna menginput nomor yang ingin diubah.
- Validasi Input: Jika nomor tidak valid, proses berhenti. Jika valid, pengguna diminta untuk menginput ulang data.
- Looping: Menggunakan perulangan untuk menginput detail layanan, harga dan kuantitas, lalu menghitung subtotal dan total bayar.
- Selesai : Selesai menandakan akhir dari flowchart.



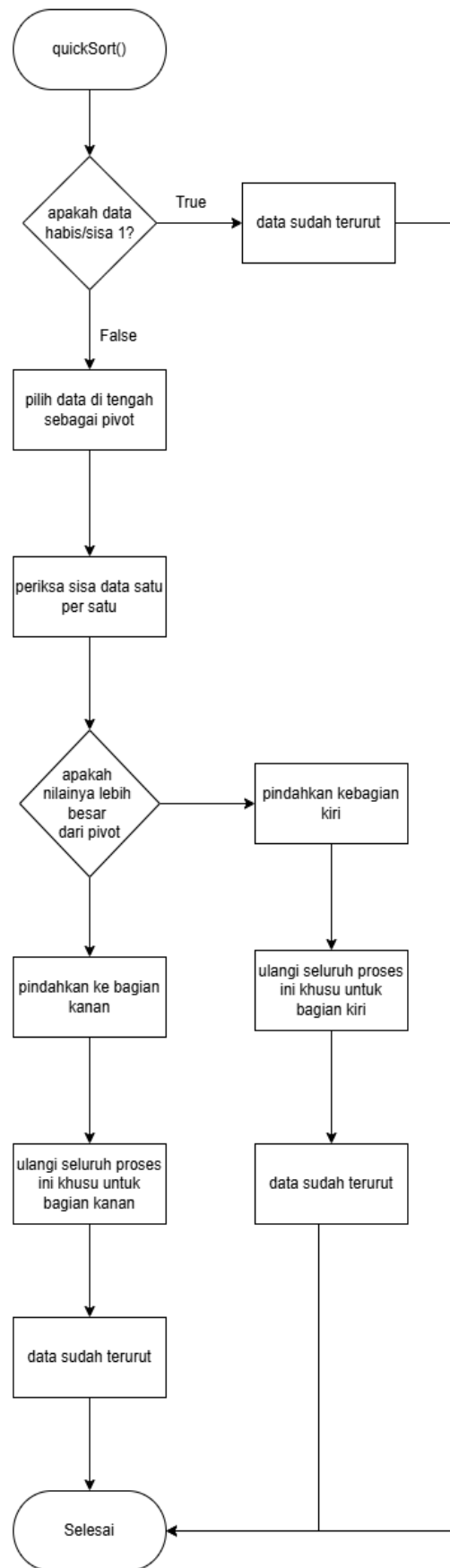
Gambar 1.6 Flowchart Fungsi Hapus.

- `hapus()`: Awal dimulainya flowchart fungsi hapus data.
- Validasi Awal: Mengecek apakah `panjang == 0`. Jika `true` maka muncul output “Belum ada data”. Jika `false` maka tampilkan data dan minta pengguna menginput nomor yang ingin dihapus.
- Algoritma Shifting: Data yang ingin dihapus ditimpa oleh data setelahnya. Ini menghilangkan data yang dipilih.
- Finalisasi: Setelah data yang dipilih dihilangkan, variabel `panjang` berkurang satu.
- Selesai : Selesai menandakan akhir dari flowchart.



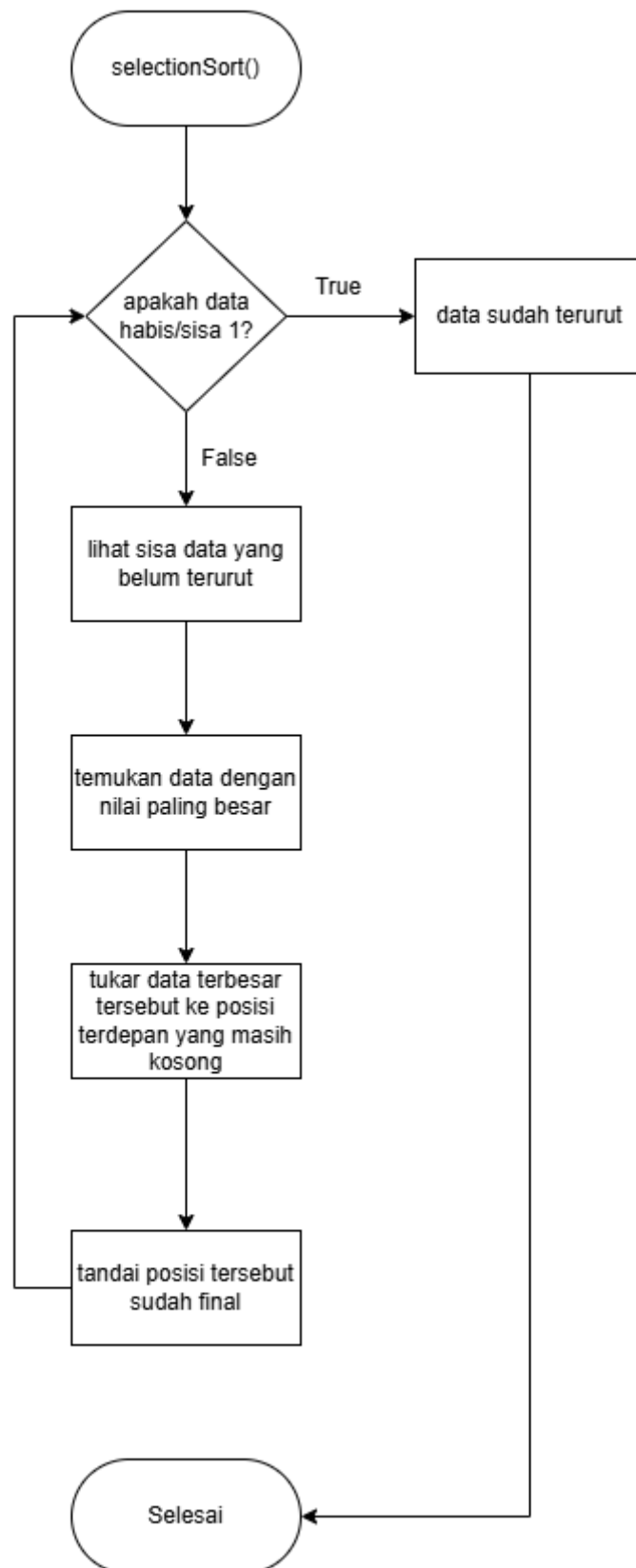
Gambar 1.7 Flowchart merge sort.

- mergeSort(): Awal dimulainya flowchart fungsi merge sort.
- Evaluasi Awal: Terima tumpukan data. Jika jumlah data di dalam tumpukan hanya 1 (atau kosong), anggap data tersebut sudah terurut. Maka flowchart selesai.
- Fase Pembelahan: Jika data lebih dari 1, belah bagian tersebut tepat di tengah menjadi dua bagian baru: kiri dan kanan.
- Rekursi: Terapkan kembali seluruh langkah algoritma ini (mulai dari langkah 1) secara spesifik untuk bagian kiri dan kanan.
- Fase Penggabungan: Siapkan satu ruang kosong. Bandingkan data teratas dari bagian kiri dan bagian kanan yang sudah terurut tadi.
- Ambil data yang lebih diprioritaskan lalu letakkan di ruang kosong.
- Ulangi langkah 5 sampai isi bagian kiri dan kanan habis dengan rapi di ruang yang baru
- Selesai : Selesai menandakan akhir dari flowchart.



Gambar 1.8 Flowchart quick sort.

- quickSort(): Awal dimulainya flowchart fungsi quick sort.
- Evaluasi Awal: Terima tumpukan data. Jika jumlah data bersisa 1 atau 0, anggap sudah terurut. Maka flowchart selesai.
- Tentukan Acuan: Pilih data di tengah sebagai pivot.
- Fase Partisi: Siapkan dua kelompok kosong: kelompok A dan kelompok B.
- Evaluasi sisa data lainnya satu per satu dan bandingkan nilainya dengan pivot:
 - Jika nilainya lebih besar dari pivot pindahkan ke kelompok A.
 - Jika tidak pindahkan ke kelompok B.
- Rekursis:
 - Terapkan kembali algoritma ini untuk kelompok A.
 - Terapkan kembali algoritma ini untuk kelompok B.
- Susun Hasil: Gabungkan hasil akhir secara berurutan dengan format: [hasil kelompok A]+[Pivot]+[hasil kelompok b].
- Selesai : Selesai menandakan akhir dari flowchart.



Gambar 1.9 Flowchart selection sort.

- `selectionSort()`: Awal dimulainya flowchart fungsi selection sort.
- Tentukan Batas: Tetapkan seluruh barisan data sebagai “Zona Belum Terurut”.
- Evaluasi Kondisi: Jika semua data sudah dipindahkan dari zona belum terurut, maka proses selesai.
- Pencarian Target: Pindai data dari ujung ke ujung hanya di dalam zona belum terurut.
- Temukan satu data dengan nilai paling tinggi.
- Fase Pertukaran: Tukar posisi data ekstrem yang baru saja ditemukan dengan data yang berada di posisi paling depan pada zona belum terurut.
- Perbarui Batas: Tandai posisi paling depan tersebut sebagai zona terurut.
- Kembali ke langkah 2 dan ulangi siklusnya sampai data habis.
- Selesai : Selesai menandakan akhir dari flowchart.

2. Deskripsi Singkat Program.

Program ini merupakan aplikasi laundry berbasis C++ yang dirancang untuk mengelola data operasional jasa pencucian secara terstruktur. Fitur utama program ini meliputi:

- Sistem Keamanan Login: Program dilengkapi dengan proteksi akses menggunakan username dan password. Pengguna diberikan batas maksimal 3 kali percobaan login, jika seluruh kesempatan gagal, program akan otomatis berhenti secara permanen sebagai langkah pengamanan.
- Pengolahan Data Transaksi (CRUD) yang berada pada fungsi/prosedur tersendiri:
 - Pencatatan Baru: Input data pelanggan beserta rincian layanan (nama layanan, harga satuan, dan berat/kuantitas).
 - Kalkulasi Otomatis: Program secara cerdas menghitung sub total per item dan mengakumulasikannya menjadi total bayar dalam satu transaksi.
 - Pembaruan & Penghapusan: Fitur untuk mengubah detail transaksi yang salah serta menghapus data dengan mekanisme pergeseran indeks array agar memori tetap tertata.
- Sistem Modular: Logika CRUD dan login dipisahkan dari fungsi main menjadi fungsi-fungsi mandiri yang saling berinteraksi.
- Penggunaan Sorting: Sorting nama pelanggan secara ascending menggunakan merge sort, sorting total bayar secara descending menggunakan quick sort, dan sorting jumlah item secara descending menggunakan selection sort.

3. Source Code

A. Fungsi Merge Sort

```
void merge(Transaksi *arr[], int l, int m, int r){
    int ukuran = r - l + 1;

    Transaksi **temp = new Transaksi *[ukuran];

    int i = l;
    int j = m + 1;
    int k = 0;

    while (i <= m && j <= r){
        if (arr[i]->nama_pelanggan <= arr[j]->nama_pelanggan){
            temp[k] = arr[i];
            i++;
        }
        else{
            temp[k] = arr[j];
            j++;
        }
        k++;
    }

    while (i <= m){
        temp[k] = arr[i];
        i++;
        k++;
    }

    while (j <= r){
        temp[k] = arr[j];
        j++;
        k++;
    }

    for (int x = 0; x < k; x++){
        arr[l + x] = temp[x];
    }

    delete[] temp;
}

void mergeSort(Transaksi *arr[], int l, int r){
    if (l < r){
        int m = l + (r - l) / 2;
```

```

        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);

        merge(arr, l, m, r);
    }
}

```

B. Fungsi Quick Sort

```

void quickSort(Transaksi *arr[], int low, int high, int panjang, int
&iterasi){
    if (low >= high)
        return;

    int mid = low + (high - low) / 2;
    double pivot = arr[mid]->total_bayar;

    int i = low, j = high;

    while (i <= j){
        while (arr[i]->total_bayar > pivot){
            i++;
        }
        while (arr[j]->total_bayar < pivot){
            j--;
        }

        if (i <= j){
            swap(arr[i], arr[j]);
            i++;
            j--;
        }
    }

    if (low < j)
        quickSort(arr, low, j, panjang, iterasi);
    if (i < high)
        quickSort(arr, i, high, panjang, iterasi);
}

```

C. Fungsi Selection Sort

```

void selectionSort(Transaksi *arr[], int panjang, int &iterasi){
    for (int i = 0; i < panjang - 1; i++){
        int max_idx = i;

        for (int j = i + 1; j < panjang; j++){
            if (arr[j]->jumlah_item > arr[max_idx]->jumlah_item){

```

```

        max_idx = j;
    }
}

    if (max_idx != i){
        swap(arr[i], arr[max_idx]);
    }
}
}

```

D. Fungsi Main

```

int main(){
    Data data;
    data.nama = "Zeinal";
    data.nim = "075";

    int panjang = 0;
    Transaksi transaksi[MAX_TRANSAKSI];

    if (login(data, 3)){
        int pilihan;
        do{
            tabel(30, "Menu Laundry");
            cout << "1. Tampilkan Data Laundry" << endl;
            cout << "2. Tambah Data Laundry" << endl;
            cout << "3. Ubah Data Laundry" << endl;
            cout << "4. Hapus Data Laundry" << endl;
            cout << "5. Sorting Nama Pelanggan Secara Ascending Menggunakan
Merge Sort" << endl;
            cout << "6. Sorting Total Bayar Secara Descending Menggunakan
Quick Sort" << endl;
            cout << "7. Sorting Jumlah Item Secara Descending Menggunakan
Selection Sort" << endl;
            cout << "8. Keluar" << endl;
            cout << "Pilihan: ";
            cin >> pilihan;

            switch (pilihan){
                case 1:
                    tampilkan(&panjang, transaksi);
                    break;
                case 2:
                    tambah(&panjang, transaksi);
                    break;
                case 3:
                    ubah(&panjang, transaksi);
                    break;
                case 4:

```

```

        hapus(panjang, transaksi);
        break;
    case 5:
        if (panjang > 0){
            Transaksi *ptrTransaksi[MAX_TRANSAKSI];
            for (int i = 0; i < panjang; i++)
                ptrTransaksi[i] = &transaksi[i];

            cout << "\n--- Sebelum Sorting ---" << endl;
            tampilkanPointer(panjang, ptrTransaksi);

            mergeSort(ptrTransaksi, 0, panjang - 1);

            cout << "\n--- Setelah Sorting ---" << endl;
            tampilkanPointer(panjang, ptrTransaksi);
        }
        else{
            cout << "Data kosong!" << endl;
        }
        break;

    case 6:
        if (panjang > 0){
            Transaksi *ptrTransaksi[MAX_TRANSAKSI];
            for (int i = 0; i < panjang; i++)
                ptrTransaksi[i] = &transaksi[i];

            cout << "\n--- Sebelum Sorting ---" << endl;
            tampilkanPointer(panjang, ptrTransaksi);

            int iterasi_qs = 0;
            quickSort(ptrTransaksi, 0, panjang - 1, panjang,
iterasi_qs);

            cout << "\n--- Setelah Sorting ---" << endl;
            tampilkanPointer(panjang, ptrTransaksi);
        }
        else{
            cout << "Data kosong!" << endl;
        }
        break;

    case 7:
        if (panjang > 0){
            Transaksi *ptrTransaksi[MAX_TRANSAKSI];
            for (int i = 0; i < panjang; i++)
                ptrTransaksi[i] = &transaksi[i];

```

```

        cout << "\n--- Sebelum Sorting ---" << endl;
        tampilkanPointer(panjang, ptrTransaksi);

        int iterasi_ss = 0;
        selectionSort(ptrTransaksi, panjang, iterasi_ss);

        cout << "\n--- Setelah Sorting ---" << endl;
        tampilkanPointer(panjang, ptrTransaksi);
    }
    else{
        cout << "Data kosong!" << endl;
    }
    break;
case 8:
    cout << "Sampai jumpa!" << endl;
    break;
default:
    cout << "Pilihan tidak valid!" << endl;
    break;
}
} while (pilihan != 8);
}
return 0;
}

```

4. Hasil Output

```
=== LOGIN ===
Username: Zeinal
Password: 000
Login Gagal. Sisa percobaan: 2
=== LOGIN ===
Username: zeinal
Password: 075
Login Gagal. Sisa percobaan: 1
=== LOGIN ===
Username: Zeinal
Password: 076
Login Gagal. Percobaan habis.
```

Gambar 4.1 Login gagal

```
=====
|      Menu Landry      |
=====
1. Tampilkan Data Landry
2. Tambah Data Landry
3. Ubah Data Landry
4. Hapus Data Landry
5. Keluar
Pilihan: 1
=====
```

No	Pelanggan	Layanan (Qty)	Sub-Total	Total Bayar
1	udin	cuci (5)	Rp25000	Rp163000
		gosok (8)	Rp48000	
		cuci&gosok (9)	Rp90000	

```
=====
```

Gambar 4.2 Output menu 1.

```

=====
|           Menu Landry           |
=====
1. Tampilkan Data Landry
2. Tambah Data Landry
3. Ubah Data Landry
4. Hapus Data Landry
5. Keluar
Pilihan: 2
Nama pelanggan: udin
Jumlah Item (maksimal 5):3
  Item ke-1 (Nama layanan): cuci
  Harga satuan: 5000
  Qty/Berat: 5
  Item ke-2 (Nama layanan): gosok
  Harga satuan: 6000
  Qty/Berat: 8
  Item ke-3 (Nama layanan): cuci&gosok
  Harga satuan: 10000
  Qty/Berat: 9
Data berhasil ditambahkan!

```

Gambar 4.3 Output menu 2.

```

=====
|           Menu Landry           |
=====
1. Tampilkan Data Landry
2. Tambah Data Landry
3. Ubah Data Landry
4. Hapus Data Landry
5. Keluar
Pilihan: 3
=====
No    Nama Pelanggan
-----
1     udin
=====
Masukkan nomor data: 1
Masukkan nama pelanggan baru: abid
Masukkan jumlah item baru (maksimal 5): 1
  Item ke-1 (Layanan): cuci
  Harga Satuan: 6000
  Qty/Berat: 7
Data laundry berhasil diubah

```

Gambar 4.4 Output menu 3.

```

=====
|           Menu Landry           |
=====
1. Tampilkan Data Landry
2. Tambah Data Landry
3. Ubah Data Landry
4. Hapus Data Landry
5. Keluar
Pilihan: 4
=====
No    Nama Pelanggan
-----
1     abid
=====
Masukkan nomor data: 1
Data Laundry berhasil dihapus

```

Gambar 4.5 Output menu 4.

```

=====
|           Menu Laundry           |
=====
1. Tampilkan Data Laundry
2. Tambah Data Laundry
3. Ubah Data Laundry
4. Hapus Data Laundry
5. Sorting Nama Pelanggan Secara Ascending Menggunakan Merge Sort
6. Sorting Total Bayar Secara Descending Menggunakan Quick Sort
7. Sorting Jumlah Item Secara Descending Menggunakan Selection Sort
8. Keluar
Pilihan: 5
--- Sebelum Sorting ---
=====
No  Pelanggan      Layanan (Qty)      Sub-Total      Total Bayar
-----
1   kdka          dkfa (8)          Rp79992        Rp142208
                   djfka (7)          Rp62216
=====
2   dkaj          dkja (9)          Rp79992        Rp79992
=====
--- Setelah Sorting ---
=====
No  Pelanggan      Layanan (Qty)      Sub-Total      Total Bayar
-----
1   dkaj          dkja (9)          Rp79992        Rp79992
=====
2   kdka          dkfa (8)          Rp79992        Rp142208
                   djfka (7)          Rp62216
=====

```

Gambar 4.6 Output menu 5.


```
=====
|           Menu Laundry           |
=====
1. Tampilkan Data Laundry
2. Tambah Data Laundry
3. Ubah Data Laundry
4. Hapus Data Laundry
5. Sorting Nama Pelanggan Secara Ascending Menggunakan Merge Sort
6. Sorting Total Bayar Secara Descending Menggunakan Quick Sort
7. Sorting Jumlah Item Secara Descending Menggunakan Selection Sort
8. Keluar
Pilihan: 6

--- Sebelum Sorting ---
=====
No  Pelanggan      Layanan (Qty)      Sub-Total      Total Bayar
-----
1   kdka          dkfa (8)          Rp79992        Rp142208
                  djfka (7)          Rp62216
=====
2   dkaj          dkja (9)          Rp79992        Rp79992
=====

--- Setelah Sorting ---
=====
No  Pelanggan      Layanan (Qty)      Sub-Total      Total Bayar
-----
1   kdka          dkfa (8)          Rp79992        Rp142208
                  djfka (7)          Rp62216
=====
2   dkaj          dkja (9)          Rp79992        Rp79992
=====
```

Gambar 4.7 Output menu 6.

```

=====
|           Menu Laundry           |
=====
1. Tampilkan Data Laundry
2. Tambah Data Laundry
3. Ubah Data Laundry
4. Hapus Data Laundry
5. Sorting Nama Pelanggan Secara Ascending Menggunakan Merge Sort
6. Sorting Total Bayar Secara Descending Menggunakan Quick Sort
7. Sorting Jumlah Item Secara Descending Menggunakan Selection Sort
8. Keluar
Pilihan: 7

--- Sebelum Sorting ---
=====
No  Pelanggan      Layanan (Qty)      Sub-Total      Total Bayar
-----
1   kdka           dkfa (8)           Rp79992         Rp142208
                   djfka (7)           Rp62216
=====
2   dkaj           dkja (9)           Rp79992         Rp79992
=====

--- Setelah Sorting ---
=====
No  Pelanggan      Layanan (Qty)      Sub-Total      Total Bayar
-----
1   kdka           dkfa (8)           Rp79992         Rp142208
                   djfka (7)           Rp62216
=====
2   dkaj           dkja (9)           Rp79992         Rp79992
=====

```

Gambar 4.8 Output menu 7.

```

=====
|           Menu Laundry           |
=====
1. Tampilkan Data Laundry
2. Tambah Data Laundry
3. Ubah Data Laundry
4. Hapus Data Laundry
5. Sorting Nama Pelanggan Secara Ascending Menggunakan Merge Sort
6. Sorting Total Bayar Secara Descending Menggunakan Quick Sort
7. Sorting Jumlah Item Secara Descending Menggunakan Selection Sort
8. Keluar
Pilihan: 8
Sampai jumpa!

```

Gambar 4.9 Output menu 8.

5. Langkah-langkah GIT

5.1 GIT Init

```
PS D:\praktikum-apl> git init
Initialized empty Git repository in D:/praktikum-apl/.git/
```

Gambar 5.1 Git Init

Menginisialisasi folder proyek menjadi repositori Git lokal.

5.2 GIT Add

```
PS D:\praktikum-apl> git add .
```

Gambar 5.2 Git add

Menambahkan file ke staging area untuk menyiapkan apa saja yang akan disimpan dalam riwayat.

5.3 GIT Commit

```
PS D:\praktikum-apl> git commit -m "Commit Pertama"
[master (root-commit) 7340d75] Commit Pertama
3 files changed, 28 insertions(+)
create mode 100644 kelas/pertemuan-1/pertemuan1.cpp
create mode 100644 kelas/pertemuan-1/pertemuan1.exe
create mode 100644 post-test/post-test-apl-1/2509106075-ZeinalAbidin-PT-1.cpp
```

Gambar 5.3 Git Commit

Mengambil atau menyimpan perubahan yang telah di-*staging* ke dalam database lokal dengan pesan penjelasan.

5.4 GIT Remote

```
PS D:\praktikum-apl> git remote add origin https://github.com/Zeinal217/praktikum-apl.git
```

Gambar 5.4 Git Remote

Menghubungkan repository lokal dengan repository di server Github.

```
PS D:\praktikum-apl> git push -u origin main
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 4 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (9/9), 18.67 KiB | 1.70 MiB/s, done.
Total 9 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Zeinal217/praktikum-apl.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Git 5.5 Git Push

Mengirimkan commit dari repositori lokal ke repositori server.